Python Engineering - Reference Guide

CLASS 14 - File I/O | Sintaxis, funciones y rutas

Esta guia es intencionadamente extensa. Sirve como referencia para volver meses despues y recordar no solo la sintaxis, sino que representa cada operacion y por que la utilizamos.

1. open()

Que hace: pide al sistema operativo acceso a un archivo y devuelve un **file object**, un objeto Python a traves del cual podemos leer o escribir.

Inputs importantes: ruta, modo y opcionalmente encoding.

Devuelve: un objeto archivo.

```
file = open('data/results.txt', 'r', encoding='utf-8')
```

Abrir un archivo no significa cargar automaticamente todo su contenido. Significa obtener un canal de comunicacion con ese recurso.

2. Modos de apertura

Modo	Significado	Si no existe	Si ya existe
'r'	Read - lectura	Error	Lee sin borrar
'w'	Write - escritura	Lo crea	Borra/reemplaza contenido
'a'	Append - anadir al final	Lo crea	Conserva y anade
'x'	Exclusive creation	Lo crea	Error si ya existe
'b'	Binary modifier	-	Se combina: 'rb', 'wb'

Regla para EX22: CSV principal con 'w', carga con 'r', log acumulativo con 'a'.

3. with open(...) as file

```
with open(file_path, 'r', encoding='utf-8') as file:
    data = file.read()

# aqui el archivo ya esta cerrado
```

`with` crea un **context manager**. Al salir del bloque, Python cierra el archivo aunque abandonemos el bloque. Evita olvidarnos de `file.close()`.

```
enter with block -> open resource
inside block     -> use resource
leave block      -> close/release resource
```

4. file.write()

Que hace: escribe texto en un archivo abierto en modo compatible con escritura. **Recibe:** un string. **Devuelve:** numero de caracteres escritos.

```
value = 3.14159
file.write(f'{value:.3f}\n')
```

En un archivo de texto convertimos el valor numerico a una representacion textual.

5. Salto de linea \n

```
file.write('first line\n')
file.write('second line\n')
```

Sin el salto, ambas escrituras quedarian pegadas.

6. file.read(), readline() y readlines()

Operacion	Que devuelve	Uso tipico
file.read()	Un string con todo lo restante	Archivos pequenos / leer todo
file.readline()	Un string con una linea	Consumir cabecera o una linea
file.readlines()	Lista de strings	Cargar todas las lineas
for line in file	Una linea por iteracion	Procesamiento linea a linea

```
header = file.readline()

for line in file:
    print(line)
```

En EX22 usamos readline() una vez para consumir la cabecera y luego iteramos por las lineas de datos.

7. strip()

Que hace: devuelve un nuevo string eliminando espacios y caracteres de borde, incluido normalmente el salto final.

```
line = '100.0,20.0,0.0\n'
clean = line.strip()
# '100.0,20.0,0.0'
```

No modifica el string original; los strings son inmutables.

8. split(separator)

Que hace: divide un string usando un separador. **Devuelve:** una lista de strings.

```
values = '100.0,20.0,0.0'.split(',')
# ['100.0', '20.0', '0.0']
```

Aunque visualmente parezcan numeros, siguen siendo strings.

9. float()

Que hace: convierte una representacion compatible a numero de coma flotante.

```
value = float('20.0')
# numeric 20.0
```

Por eso el loader sigue texto -> split -> strings -> float.

10. encoding='utf-8'

Un archivo de texto guarda bytes. El encoding define como traducimos caracteres entre texto Python y bytes. UTF-8 es una eleccion moderna y general.

11. pathlib.Path

Path representa rutas como objetos en lugar de tratarlas como strings arbitrarios.

```
from pathlib import Path

path = Path('data') / 'vertical_trajectory.csv'
```

El operador / aqui no divide numeros: Path lo redefine para unir partes de una ruta.

12. __file__

__file__ es una variable especial del modulo que contiene la ruta del archivo Python actual.

```
base_dir = Path(__file__).resolve().parent
```

Expresion	Significado
Path(__file__)	Convierte la ruta del script en objeto Path
.resolve()	Obtiene una ruta absoluta normalizada
.parent	Obtiene la carpeta que contiene el archivo

13. Relative path vs absolute path

```
# relative
Path('data/results.csv')

# absolute example
Path('/home/user/project/data/results.csv')
```

Una ruta relativa necesita un punto de referencia. En VS Code suele depender del **current working directory**, no necesariamente de la carpeta del script. Por eso EX22 ancla la ruta a __file__.

14. Path.mkdir()

```
data_dir.mkdir(exist_ok=True)
```

Crea la carpeta. `exist_ok=True` significa que si ya existe no se considera un error.

15. CSV: Comma-Separated Values

CSV no es una matriz NumPy ni una hoja Excel. Es un **archivo de texto** con una convencion de columnas separadas.

```
h_m,v_m_s,t_s
100.000000,20.000000,0.000000
102.000000,19.019000,0.100000
```

Ventajas: simple, legible e intercambiable. Limitacion: al leer debemos reconstruir los tipos.

16. np.array(rows)

Durante la lectura construimos primero una lista de filas y al final la convertimos a NumPy.

```
rows = [[100.0,20.0,0.0], [102.0,19.019,0.1]]
trajectory = np.array(rows)
```

Separamos parsing textual y estructura numerica.

17. np.allclose()

Comprueba igualdad aproximada elemento a elemento dentro de tolerancias.

```
np.allclose(original, loaded)
# True
```

Es apropiado porque el viaje por texto decimal puede introducir pequenas diferencias de representacion.

18. Round-trip validation

```
ORIGINAL NUMBERS
 -> WRITE / SERIALIZE
 -> FILE
 -> READ / PARSE
 -> RECOVERED NUMBERS
 -> COMPARE
```

No comprobamos solo que exista un archivo: comprobamos que la informacion pueda recuperarse.

19. Responsabilidad de cada modulo

Modulo	Responsabilidad
vertical_state.py	Modelo fisico/estado y comportamiento
file_io.py	Datos Python <-> representacion en archivo
main.py	Orquestar el flujo de la aplicacion
data/	Datos persistentes generados

Esto es **separation of concerns**: cada parte tiene una responsabilidad principal.

20. Cheat sheet final

```
open(path, 'r')  -> read
open(path, 'w')  -> write/replace
open(path, 'a')  -> append

with open(...) as file: -> automatic close
file.write(str)          -> write text
file.readline()          -> one line
line.strip()             -> trim edges/newline
line.split(',')          -> list of strings
float(text)              -> text to float

Path(__file__).resolve().parent -> script directory
data_dir.mkdir(exist_ok=True)   -> ensure folder
np.allclose(a, b)               -> approximate check
```